Project 13 (Bonus) — CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass

Type: Multi-agent (1 Supervisor + 3 reviewers) · **Stack:** Python, LangGraph (`Command`), LangChain, RAG (Chroma), OpenAI SDK, Pydantic (+ optional FastAPI/Docker) · **Level:** Advanced (a test of what you've learned)

Tagline: A Supervisor-led team of three specialist reviewers — Security, Style (grounded in your real style guide via RAG), and Test-Coverage — that turns a raw code diff into one compiled, de-duplicated, severity-sorted review comment.

> Not part of the numbered Doc01-19 arc — the capstone bonus. It teaches no new ideas on purpose: every skill was taught in Docs 01-19 and proven in Projects 1-12. It exists to prove the design **generalizes** — the same Supervisor + specialists (Doc11), the same RAG (Doc08), the same production layer (Doc12-13) — on a completely different problem: reviewing code instead of writing content.

Charter (what this project is)

Build a PR-review team: a **Supervisor** routes a diff to whichever reviewers actually apply (a docs-only change skips Security and Test-Coverage; a config-only change skips Style), three **specialist reviewers** each do one narrow job, and the Supervisor **compiles** their findings into one clean comment — de-duplicated and sorted by severity — not three reports stapled together. The Style reviewer is grounded in your **real style guide** through RAG, so it checks your actual rules, not what the model happens to "know."

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4

The Story — what you're actually building

A pull request lands. Before a human opens it, three specialists already looked it over: one for **security** (secrets left in code, unsafe patterns like `eval`), one for **style** against your team's *actual* guide (not generic advice — your real document, searched), one for **test coverage** (did the changed code get any tests?). A fourth — the lead **Supervisor** — reads all three, drops overlap and noise,

sorts by severity, and writes one clear comment: here's what's worth your attention. That's CodeGuard — the ContentForge (Project 4) shape, pointed at code review.

The point isn't the review; it's proving the patterns from Projects 1-12 generalize. A Supervisor that routes to the right specialist, specialists that stay narrow and grounded (Style uses RAG, not freelancing) — that shape recurs everywhere: moderation, compliance, multi-criteria approvals.

> **Before you read further — think about it yourself:** which reviewers should a `.md`-only diff run, and which should a `config.yaml`-only diff run? And why ground the Style reviewer in your real guide with RAG instead of just prompting "follow good style"? Settle both before Step 1.

Project Structure (the professional layout)

`main.py` at the repo root, all code in `src/`. The style guide and sample diffs live in `data/`.

The final layout (after Step 3; Step 4 adds the API)

```
<repo folder>/   (the CodeGuard repo — full name in the heading above)
├── data/
│   ├── sample_diffs/           a few .diff files (clean, secret, docs-only)
│   └── style_guide.md          your real style rules (RAG source)
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── coverage_reviewer.py  run_coverage() -> findings
│   │   ├── security_reviewer.py  run_security() -> findings
│   │   ├── style_reviewer.py     run_style() -> findings (RAG + LLM)
│   │   └── supervisor.py         choose_reviewers/decide_next/compile_review
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── ingest.py             build_style_store()
│   │   ├── retriever.py          retrieve_rules()
│   │   └── store.py              get_collection()
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   └── style.txt
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
```

```
│   ├── __init__.py
│   ├── config.py
│   ├── diff.py                  parse_diff(), categorize()
│   ├── exceptions.py
│   ├── graph.py                 build_graph(model, style_collection)
│   ├── schemas.py               Finding
│   └── state.py                 ReviewState, start_state()
├── tests/
│   ├── integration/
│   │   └── test_live_review.py   real model + RAG
│   └── unit/
│       ├── test_compile.py
│       ├── test_config.py
│       ├── test_diff.py
│       ├── test_routing.py
│       └── test_security.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds config/exceptions/logger, `models/llm`, `src/diff.py`, `src/schemas.py` (`Finding`), `agents/security_reviewer.py`, `main.py`, `test_config`, `test_diff`, `test_security`.
- **Step 2** adds `src/rag/` (over the style guide) and `agents/style_reviewer.py`.
- **Step 3** adds `agents/coverage_reviewer.py`, `agents/supervisor.py` (routing + compile), `state.py`, `graph.py`, `test_routing`, `test_compile`.
- **Step 4 (optional)** wraps it in a FastAPI `/review` endpoint + a routing eval (reusing Project 5).

Setup (do this once, before Step 1)

```
cd CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install openai langchain-openai langgraph chromadb python-dotenv pydantic
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty

`__init__.py`. Write a short `data/style_guide.md` with at least one **specific, unusual** rule (so a model couldn't invent it from general knowledge — that's what proves RAG is working).

A Real Example (so this isn't just theory)

- A code diff with a hardcoded secret, an `eval(...)`, a `camelCase` function name, and no test changes → the review flags HIGH (secret), MEDIUM (eval, no tests), LOW (the camelCase style rule, cited from your guide).
- A `README.md`-only diff → Security and Test-Coverage are **skipped**; only Style runs.
- A `config.yaml`-only diff → Style is skipped (nothing routes, or only applicable reviewers run).
- A clean code diff *with* a test change → "no issues found."

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc08/11, not pytest):

- **`tests/unit/`** — no model: `test_config`; `test_diff` (parsing added lines); `test_security` (the scanner on planted + clean diffs); `test_routing` (which reviewers per file type); `test_compile` (dedup + severity sort).
- **`tests/integration/test_live_review.py`** — the real model + RAG: the Style reviewer cites the right guide rule; the full graph compiles one review; a docs-only diff skips Security.

Each step ends with a **Tests for this step** box.

Step 1 — The Security Reviewer on a Diff

What this step does: the Security reviewer, working on a diff, proven on 2-3 example diffs (including a clean one) before anything else exists.

Why this step matters: prove the smallest piece — and a reviewer only ever tested on bad input will happily flag good code, so test a clean diff too.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write config/exceptions/ `utils/logger`, `models/llm`, `src/schemas.py` (`Finding`: severity, category, message), and `src/diff.py` (`parse_diff(text)` → a list of `{path, added}` for each changed file — **only the added lines**, like Doc08's chunking of what's relevant). 2. Write `agents/security_reviewer.py`: `run_security(files)` — scan each file's added lines for secret patterns (hardcoded keys/passwords, AWS keys) and unsafe patterns (`eval(`, `subprocess(..., shell=True)`, `pickle.loads`) and return `Finding`s. (A deterministic scanner is the trustworthy core; you can add an LLM pass for judgment calls.) 3. Write `main.py` to run it on example diffs (including a clean one) and print the findings.

Your files after Step 1: config, exceptions, `utils/logger`, `models/llm`, `schemas`, `diff`, `agents/security_reviewer`, `main.py`, `tests/unit/{test_config,test_diff,test_security}.py`.

Tests for this step: `test_diff` (added lines parsed); `test_security` (secret + `eval` flagged; a clean diff flags nothing); `test_config`.

Step 2 — The Style Reviewer, Grounded in Your Style Guide With RAG

What this step does: the Style reviewer, grounded in your real style guide through RAG — it searches your document, not general training knowledge.

Why this step matters: an ungrounded Style reviewer looks correct until it confidently cites a rule that isn't yours. RAG makes that failure checkable: the finding cites the actual retrieved rule.

What's new vs. Step 1: `src/rag/` and `agents/style_reviewer.py`. **What stays the same:** the Security reviewer.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/rag/` (`get_collection`, `build_style_store` — chunk the style guide into rules and embed them, `retrieve_rules(collection, query)`). 2. Write `agents/style_reviewer.py`: `run_style(model, collection, files)` — for the changed lines, retrieve the relevant rules and ask the model (structured output) which lines violate which rule, returning `Finding`s that **cite the retrieved rule**. 3. Put a specific, unusual rule in your guide and test that the Style reviewer catches a violation of *that* rule (a generic rule proves nothing about retrieval).

Your files after Step 2: + `src/rag/*`, `agents/style_reviewer.py`, `src/prompts/style.txt`.

Tests for this step: the integration test — the Style reviewer cites the specific guide rule for a matching diff.

Step 3 — The Test-Coverage Reviewer, the Supervisor (Routing + Compile) — final 4 agents

What this step does: finishes the team — the Test-Coverage reviewer, and the Supervisor that routes by diff and compiles findings into one clean review.

Why this step matters: three separately-correct reviewers stuck together is not one useful review. This is where the multi-agent value — routing to skip irrelevant reviewers, and compiling (dedup + sort) — either shows up or doesn't.

What's new vs. Step 2: `agents/coverage_reviewer.py`, `agents/supervisor.py`, `state.py`, `graph.py`. **What stays the same:** the reviewers.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `agents/coverage_reviewer.py`: `run_coverage(files)` — flag a changed code file (`.py` / `.js` / `.ts`, not a test) when the diff includes no test changes. 2. Write `agents/supervisor.py`: `choose_reviewers(files)` (route by file category — code → all; docs-only → skip Security/Coverage; config-only → skip Style), `decide_next(state)` (the next pending reviewer, else compile), and `compile_review(findings)` (de-dup by (category, message), sort by severity, format one comment). 3. Write `state.py` (`ReviewState`) and `graph.py`: `START → supervisor`; each reviewer node runs its reviewer and returns `Command(goto="supervisor", ...)`; the supervisor routes to the next pending reviewer or to `compile → END`.

Your files after Step 3: + `agents/coverage_reviewer.py`, `agents/supervisor.py`, `state.py`, `graph.py`, `tests/unit/{test_routing,test_compile}.py`.

Tests for this step: `test_routing` (code → all; docs-only → Style only; config-only → none/Style skipped); `test_compile` (dedup removes a repeat; HIGH sorts before MEDIUM; empty → "no issues found").

Step 4 — Production Wrap (optional)

What this step does: wraps the 4-agent system in Project 5's production layer — a FastAPI `/review` endpoint, a test suite of example diffs with known findings, and Docker — proving that layer generalizes too.

Why this step matters: the difference between a project you can describe and one you can hand someone a link to. Reuse Project 5 almost unchanged.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Add `src/pipeline.py` (`review_diff(model, collection, diff) -> str`: build the graph, run it, return the compiled review) and a FastAPI `/review` route (`POST {diff}` → the review) + `/health` (reuse Project 5's shape). 2. Add an eval that specifically checks **routing** (e.g. a docs-only diff must *not* produce a Security finding) — a suite that only checks reviewer accuracy could pass even if routing were broken. 3. (Optional) a Dockerfile, as in Project 5.

Final Deliverable: a Supervisor-led team of 3 specialist reviewers that turns a code diff into one compiled, de-duplicated, severity-sorted review comment, with Style grounded in your real guide through RAG.

Run, Test and Configure the Final Project

```
python main.py
python -m tests.unit.test_config
python -m tests.unit.test_diff
python -m tests.unit.test_security
python -m tests.unit.test_routing
python -m tests.unit.test_compile
python -m tests.integration.test_live_review
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity

Checklist Before You Call This Done

- [ ] 3 independent reviewers, each tested alone (including on a clean diff) before joining the team
- [ ] The Style reviewer cites a rule from your real, searched style guide — not general knowledge
- [ ] The Supervisor routes by the diff — not every reviewer runs on every diff
- [ ] The final output is one compiled, de-duplicated, severity-sorted review — not three reports stuck together
- [ ] You can explain why the same Supervisor+specialists+RAG shape fits code review as well as it fit ContentForge

Publish It to GitHub

Name the repo `CodeGuard-Multi-Agent-PR-Review-Security-Style-And-Test-Coverage-In-One-Pass`, add the `README.md`, commit `.env.example`, push. It's a second, different multi-agent system in a domain any interviewer understands — proof the architecture generalizes.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** — but try harder here before asking than on Projects 1-5. That's the actual exercise.